

天工2.0 Pro版SDK(ROS2)文档

适用软件版本	v2.0.5.2 及以后版本
文档更新时间	2026-03-30

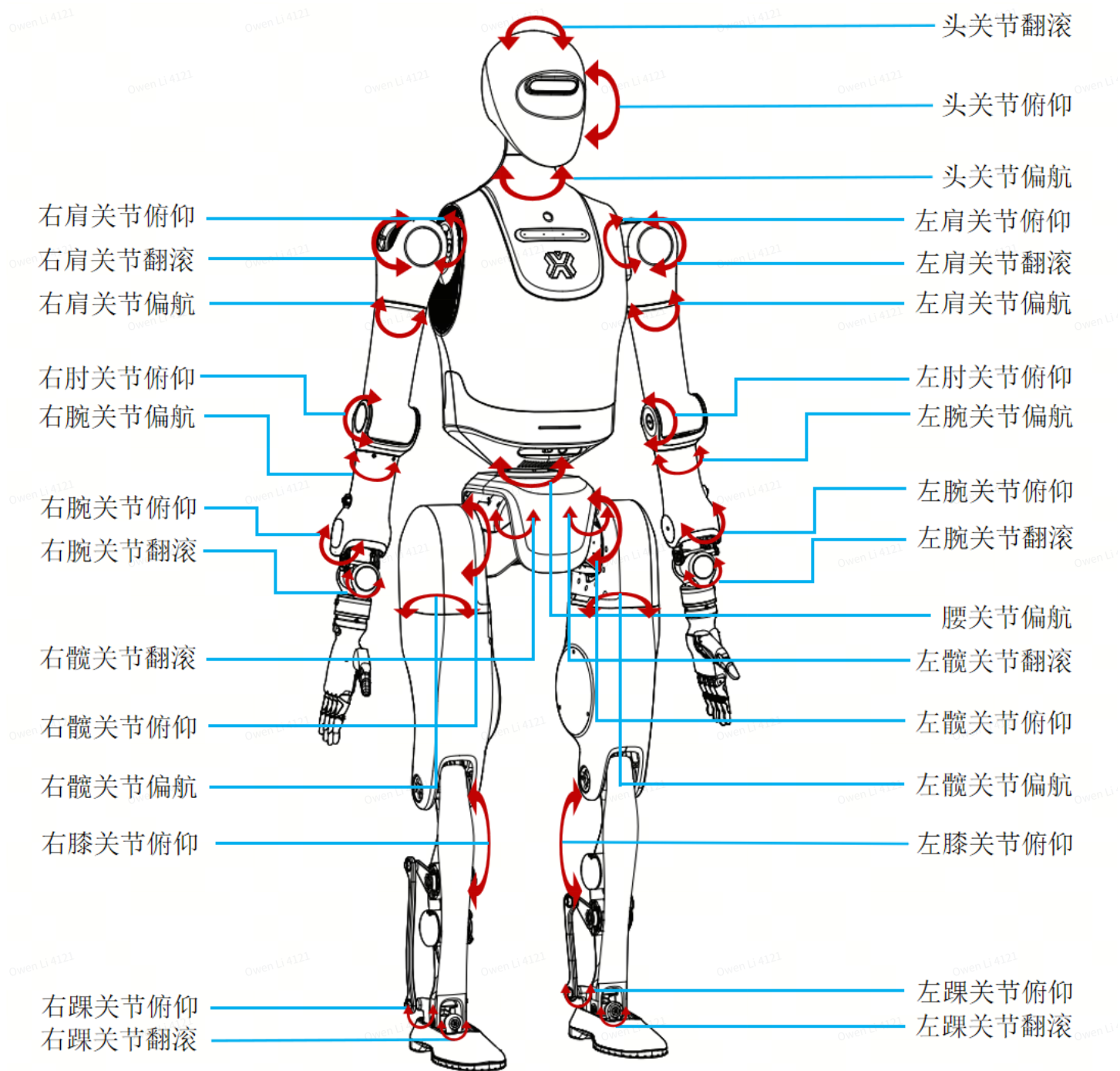
1. 关于天工2.0 Pro版

- 天工是北京人形机器人创新中心研发的全尺寸纯电驱拟人奔跑机器人，具备高度仿生的躯干构型和拟人化的运动控制能力。整机共拥有 42 个自由度，双臂采用七自由度机械臂设计，手腕搭载高精度六维力传感器，并配备灵巧五指双手。
- 天工可实现站立、行走、奔跑及多个特定动作，能够稳定适应楼梯、斜坡、沙地等复杂地形，具备优异的动态平衡与抗干扰能力。搭载强劲关节驱动、高精度 IMU 与双电池快换系统，整机续航时间超过 3.5 小时。
- 此外，天工集成语音模组与语音交互大模型，支持自然语言控制；适配“慧思开物”大模型，具备多项特定场景操作演示 demo，支持遥操作与数据采集，整机算力可达 550 TOPS。
- 作为通用人形机器人母平台，天工开放全身关节及各传感器控制接口，便于行业二次开发，广泛应用于科研、教育及工业场景。

1.1 功能特点

- 复杂地形自适应：**可平稳行走于楼梯、斜坡、沙地等不规则地形；
- 拟人化奔跑：**具备稳定奔跑能力；
- 动态平衡控制：**能实时抗外部干扰，维持姿态稳定；
- 两种运控模式：**已部署传统运控模式和强化学习运控模式；
- 语音交互控制：**搭载语音模组与语音大模型，支持自然语音指令与交互；
- 开放式接口设计：**提供全身关节与传感器控制接口，适配“慧思开物”大模型，支持遥操作、数据采集与二次开发。

1.2 全身关节参数

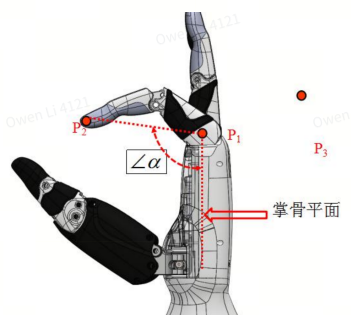
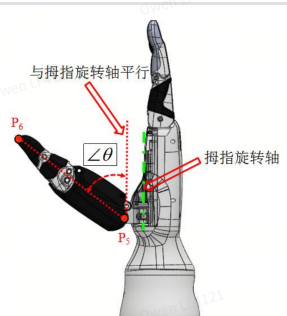
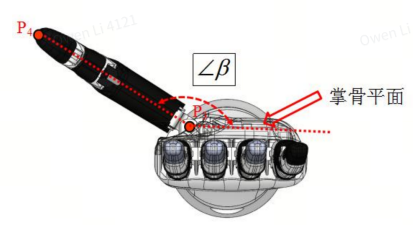


所属部位	名称	运动范围 (°)	扭矩系数 (Nm/A)	最大转速 (rpm)	KP/KD范围	额定力矩 (Nm)	峰值力矩 (Nm)	关节名称
头部	头关节翻滚 Head Roll	-26°~+26°	2.16	64	Kp: 0~2000 kd: 0~300	3	9	1
	头关节俯仰 Head Pitch	-25°~+25°	2.16	64	Kp: 0~2000	3	9	2

					kd: 0~300			
	头关节偏航 Head Yaw	-90°~+90°	2.16	64	Kp: 0~2000 kd: 0~300	3	9	3
腰部	腰关节偏航 Waist Yaw	-170°~+17 0°	3.207	88	Kp: 0~2000 kd: 0~300	35	91	31
左臂	左肩关节俯仰 Left Shoulder Pitch	-170°~+17 0°	3.207	88	Kp: 0~2000 kd: 0~300	35	91	11
	左肩关节翻滚 Left Shoulder Roll	-150°~+15°	3.37	120	Kp: 0~2000 kd: 0~300	20	60	12
	左肩关节偏航 Left Shoulder Yaw	-170°~+17 0°	2.43	73	Kp: 0~2000 kd: 0~300	10	30	13
	左肘关节俯仰 Left Elbow Pitch	-150°~+15°	2.43	73	Kp: 0~2000 kd: 0~300	10	30	14
	左腕关节偏航 Left Wrist Yaw	-170°~+17 0°	1.71	146	Kp: 0~2000 kd: 0~300	5	15	15
	左腕关节俯仰 Left Wrist Pitch	-45°~+60°	2.16	72	Kp: 0~2000 kd: 0~300	3	9	16
	左腕关节翻滚 Left Wrist Roll	-95°~+75°	2.16	72	Kp: 0~2000 kd: 0~300	3	9	17

右臂	右肩关节俯仰 Right Shoulder Pitch	-170°~+170°	3.207	88	Kp: 0~2000 kd: 0~300	35	91	21
	右肩关节翻滚 Right Shoulder Roll	-15°~+150°	3.37	120	Kp: 0~2000 kd: 0~300	20	60	22
	右肩关节偏航 Right Shoulder Yaw	-170°~+170°	2.43	73	Kp: 0~2000 kd: 0~300	10	30	23
	右肘关节俯仰 Right Elbow Pitch	-150°~+15°	2.43	73	Kp: 0~2000 kd: 0~300	10	30	24
	右腕关节偏航 Right Wrist Yaw	-170°~+170°	1.71	146	Kp: 0~2000 kd: 0~300	5	15	25
	右腕关节俯仰 Right Wrist Pitch	-45°~+60°	2.16	72	Kp: 0~2000 kd: 0~300	3	9	26
	右腕关节翻滚 Right Wrist Roll	-75°~+95°	2.16	72	Kp: 0~2000 kd: 0~300	3	9	27
腿部	髋关节翻滚 Hip Roll	-45°~+45°	2.3	125	Kp: 0~2000 kd: 0~300	/	124	左51 右61
	髋关节俯仰 Hip Pitch	-160°~+120°	2	125	Kp: 0~2000 kd: 0~300	/	300	左52 右62
	髋关节偏航 Hip Yaw	-60°~+60°	2.3	125	Kp: 0~2000	/	124	左53 右63

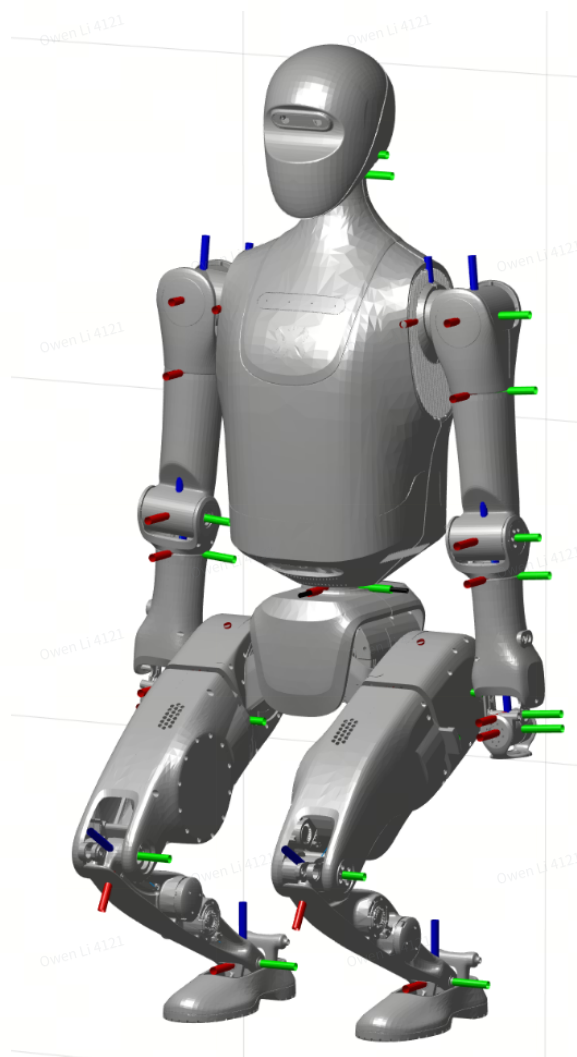
				kd: 0~300			
膝关节俯仰 Knee Pitch	0°~+137°	2	125	Kp: 0~2000 kd: 0~300	/	300	左54 右64
踝关节俯仰 Ankle Pitch	-70°~+30°	2.6	128	Kp: 0~2000 kd: 0~300	/	36	左55 右65
踝关节翻滚 Ankle Roll	-30°~+30°	2.6	128	Kp: 0~2000 kd: 0~300	/	36	左56 右66

角度	运动范围 (°)	图例说明
小拇指 无名指 中指 食指	+19°~+176.7°	
大拇指弯曲角度	-13°~+53.6°	
大拇指旋转角度	+90°~+165°	

1.3 整机urdf，坐标系，关节旋转轴与关节零点

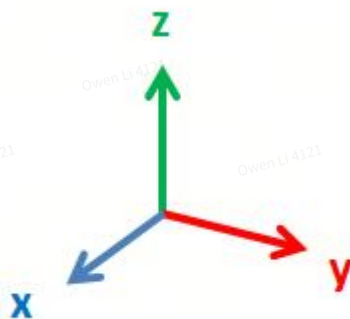
1.3.1 关节零点

如下图所示：



1.3.2 坐标系

- a. 整机坐标系-X轴（Roll）以机器人前方为正方向；
- b. 整机坐标系-Y轴（Pitch）以机器人左方为正方向；
- c. 整机坐标系-Z轴（Yaw）以机器人上方为正方向；

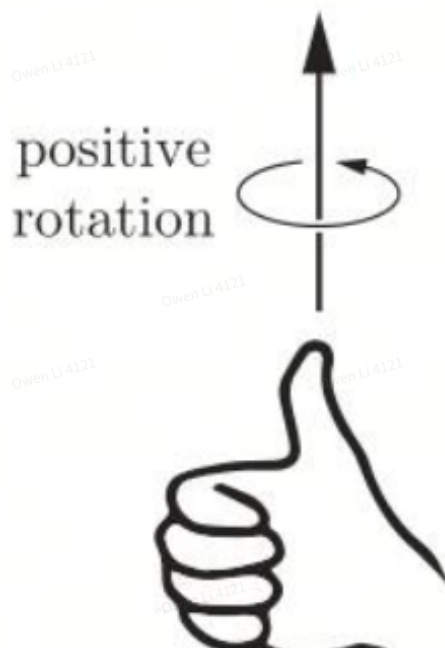
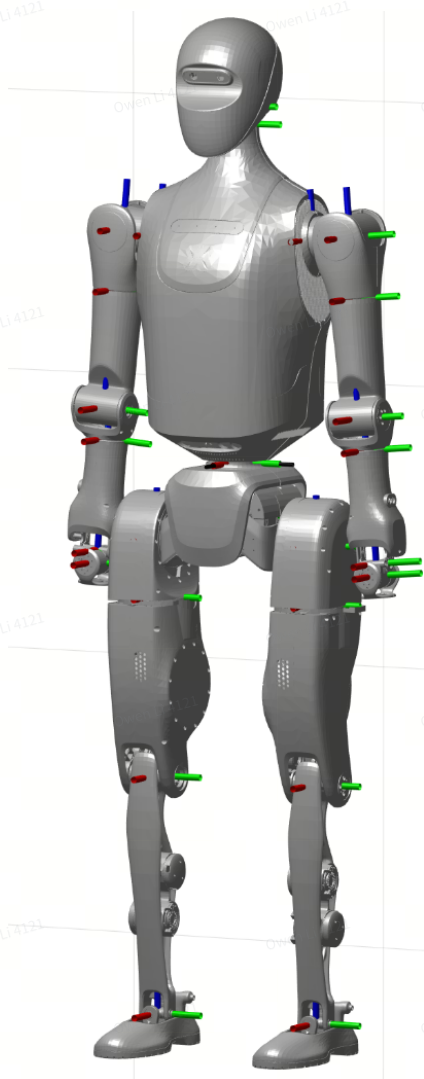


1.3.3 旋转轴 & 正方向

- a. 机器人每个旋转轴的轴线，按照下图的状态为基准，和机器人整机坐标系的正方向对应去寻找旋转轴的正方向；

b. 举例：

- i. 比如腰部关节的轴线是竖直的，和整机Z轴（Yaw）靠近，所以腰部关节的正方向是向上，同构右手定则能够确认关节的选装正方向；
- ii. 比如腿部髋关节Pitch的轴线，和整机Y轴（Pitch）靠近，所以髋关节pitch的正方向是向左，同构右手定则能够确认关节的选装正方向。



2. 快速操作指南

⚠注意：机器人目前有两种启动方式：自启动和手动程序启动。

如启用自启动，则✅需执行步骤 3.d. 按A键自检，❌无需执行步骤 5. 程序启动；

如关闭自启动，手动程序启动，则❌无需执行步骤 3.d. 按A键自检，✅需执行步骤 5. 程序启动。

1. 确认机器人悬吊于保护支架之上；

2. 将遥控器上的电源键先短按后立即长按，打开遥控器；

3. 开机上电：

a. 按下总开关键；

- b. 短按开/关机键；
- c. 顺时针旋转急停按钮直至弹出；
- d. **（自启动启用的前提下）** 等待机器人开机完成后（即开机音效结束），在 F 拨杆置于中间挡位的前提下，按遥控器 A 键启动自检，自检成功后（即整机状态灯变为蓝绿常亮状态）进入可操作的Ready状态。

4. 配置Wi-Fi步骤：

- a. 用以太网线连接机器人背后的调试以太网接口和用户电脑端；
- b. 用户电脑配置以太网端口地址为192.168.41.xx/255.255.255.0（不要配置成192.168.41.1、192.168.41.2、192.168.41.3）；
- c. 打开终端中输入 `ssh ubuntu@192.168.41.x` 登录需要配置的主控板（运控x86、大模型orin、导航orin分别为192.168.41.1、192.168.41.2、192.168.41.3）；
- d. 在上一步的终端中继续依次输入如下命令配置IP：

代码块

```
1 sudo nmcli device wifi list #查看Wi-Fi热点的SSID
2 sudo nmcli device wifi connect 'Wi-Fi SSID' password 'Wi-Fi密码' #连接Wi-Fi
3 ifconfig #检查是否连接成功
```

- e. Wi-Fi配置完成，拔掉网线。

• Wi-Fi配置完成的前提下，每次机器人无需再配置Wi-Fi，直接用Wi-Fi连接的步骤如下：

- a. 输入 `ifconfig`，查看给主控配置的IP地址；
- b. 输入 `ssh 用户名@x.x.x.x` 登录连接主控板服务器，其中 `x.x.x.x` 为上一步获取的IP地址。随后可进行下一步程序启动。

5. **（自启动关闭的前提下）** 程序启动

- a. 手动启动请先确保已经关闭自启动功能：

代码块

```
1 sudo systemctl stop proc_manager.service
```

- b. 首先，打开第一个终端，依次输入如下命令启动**主控节点**：

代码块

```
1 sudo su
2 cd ros2ws
3 source install/setup.bash
```

```
4 ros2 launch body_control body.launch.py
```

c. 其次，打开第二个终端，如需启动**运控-传统算法**节点则依次输入如下命令：

代码块

```
1 sudo su
2 cd ros2ws
3 source install/setup.bash
4 ros2 launch motion_control motion.py
```

d. 如需启动**运控-强化学习**节点则应改为依次输入如下命令：

代码块

```
1 sudo su
2 cd ros2ws
3 source install/setup.bash
4 ros2 run rl_control rl_control_node
```

此时可以开始使用 SDK 进行开发调试。

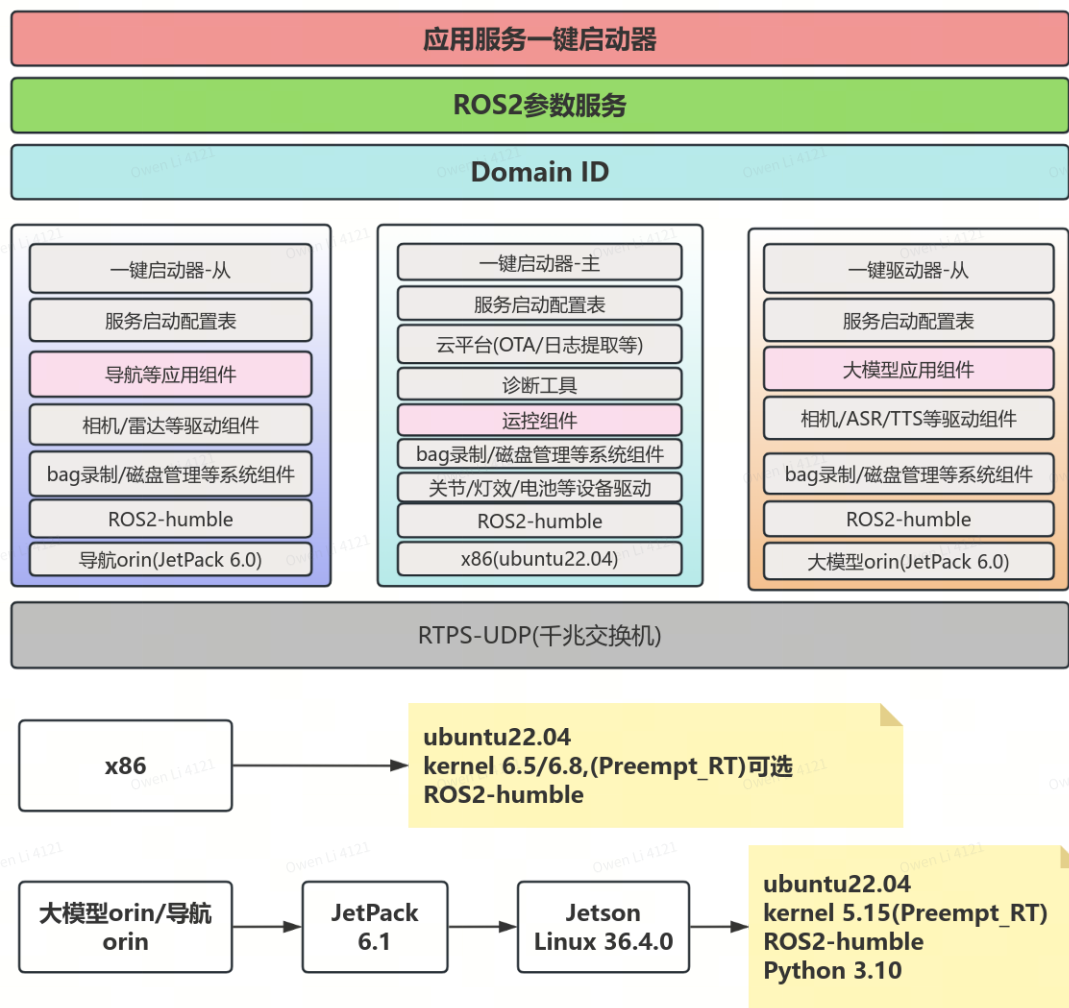
6. 关机：

- 确认机器人已停止并返回站立状态；
- 按下遥控器上的“C”键使机器人僵停；
- 将机器人固定在支架上，并向上吊起；
- 按下急停按钮；
- 长按开/关机键6秒；
- 按下总开关键，此时所有状态灯全部熄灭；
- 将遥控器上的电源键先短按后立即长按，关闭遥控器。

3. SDK 概述

具身天工机器人2.0 Pro ROS2.0 SDK由北京人形机器人创新中心开发，提供了丰富的接口，涵盖各部位电机控制、IMU（惯性测量单元）、语音、电池、遥控器等。可用于编写和部署机器人应用程序，旨在帮助开发人员快速灵活地构建自己的应用程序来精确控制和使用机器人，以满足在不同应用场景下的需求。

3.1 基本框架 - ROS2



3.2 系统环境依赖

为了确保最佳的开发体验和兼容性，建议在Ubuntu 22.04系统下进行开发。当前暂不支持在Mac和Windows系统下开发。

3.3 运行环境说明

3.3.1 整体参数配置路径

代码块

```
1 #x86本体安装路径
2 /home/ubuntu/ros2ws
3 #x86本体msg定义
4 /home/ubuntu/ros2ws/install/bodyctrl_msgs/share/bodyctrl_msgs
5
6 #x86全局配置参数路径
7 /home/ubuntu/data/param
```

```
8 #orin全局配置参数路径
9 /home/nvidia/data/param
```

3.3.2 配置参数及环境变量

- 配置参数位置: `/home/ubuntu/data/param/ros2_config.txt`

代码块

```
1 ROS_LOG_DIR=/home/ubuntu/logs
2 ROS2_BAG_DIR=/home/ubuntu/bags
3 FASTRTPS_DEFAULT_PROFILES_FILE=/home/ubuntu/data/param/dds_profile.xml
4 RMW_IMPLEMENTATION=rmw_fastrtps_cpp
5 ROS_DOMAIN_ID=0
```

注:

ROS_LOG_DIR: 日志保存位置

ROS2_BAG_DIR: bag包保存位置

FASTRTPS_DEFAULT_PROFILES_FILE: dds配置文件, 默认禁止使用wifi调试dds通信;

RMW_IMPLEMENTATION: 使用的中间件类型

ROS_DOMAIN_ID: domain id

- 环境变量: `/home/ubuntu/data/param/ros2_setup.bash`

上面的配置参数在登录终端时自动导出为环境变量, 也可执行以下命令手动导出

代码块

```
1 source /home/ubuntu/data/param/ros2_setup.bash
```

4. 全局值说明

4.1 全身关节id定义

- 消息名称: `MotorName`
- 数据定义位置: `MotorName.msg`
- 说明: Pro版天工头部、臂部、腰部和腿部的电机的命名ID定义如下。

```
1  # head 00 + id
2  uint16 MOTOR_HEAD_1 = 1
3  uint16 MOTOR_HEAD_2 = 2
4  uint16 MOTOR_HEAD_3 = 3
5
6  # arm-left 10 + id
7  uint16 MOTOR_ARM_LEFT_1 = 11
8  uint16 MOTOR_ARM_LEFT_2 = 12
9  uint16 MOTOR_ARM_LEFT_3 = 13
10 uint16 MOTOR_ARM_LEFT_4 = 14
11 uint16 MOTOR_ARM_LEFT_5 = 15
12 uint16 MOTOR_ARM_LEFT_6 = 16
13 uint16 MOTOR_ARM_LEFT_7 = 17
14
15 # arm-right 20 + id
16 uint16 MOTOR_ARM_RIGHT_1 = 21
17 uint16 MOTOR_ARM_RIGHT_2 = 22
18 uint16 MOTOR_ARM_RIGHT_3 = 23
19 uint16 MOTOR_ARM_RIGHT_4 = 24
20 uint16 MOTOR_ARM_RIGHT_5 = 25
21 uint16 MOTOR_ARM_RIGHT_6 = 26
22 uint16 MOTOR_ARM_RIGHT_7 = 27
23
24 # body (including waist) 30 + id
25 uint16 MOTOR_WAIST_1 = 31
26
27 # 40 rfu
28
29 # leg-left 50 + id
30 uint16 MOTOR_LEG_LEFT_1 = 51
31 uint16 MOTOR_LEG_LEFT_2 = 52
32 uint16 MOTOR_LEG_LEFT_3 = 53
33 uint16 MOTOR_LEG_LEFT_4 = 54
34 uint16 MOTOR_LEG_LEFT_5 = 55
35 uint16 MOTOR_LEG_LEFT_6 = 56
36
37 # leg-right 60 + id
38 uint16 MOTOR_LEG_RIGHT_1 = 61
39 uint16 MOTOR_LEG_RIGHT_2 = 62
40 uint16 MOTOR_LEG_RIGHT_3 = 63
41 uint16 MOTOR_LEG_RIGHT_4 = 64
42 uint16 MOTOR_LEG_RIGHT_5 = 65
43 uint16 MOTOR_LEG_RIGHT_6 = 66
```

5. 接口说明

5.1 产品序列号

5.1.1 命令行读取

注：仅在 x86 有效。

代码块

```
1 $ xsn
2 00TG20LD25250001
```

5.1.2 ROS 接口

1. ROS Srv 定义

代码块

```
1 # xsys_msgs/srv/GetSerialNumber.srv
2
3 string SERVICE_NAME = /xsys/get_serial_number
4
5 ---
6 string serial_number
```

2. package 配置

代码块

```
1 <package format="3">
2   <depend>xsys_msgs</depend>
3 </package>
```

3. CMake 配置

代码块

```
1 find_package(xsys_msgs REQUIRED)
2
3 ament_target_dependencies(
4   <your_target>
5   xsys_msgs
6 )
```

4. Setup

x86 的 ros2ws 和 orin 的 lyre_ros2 提供了 `xsys_msgs` 包。在构建或者运行自己的 ROS 程序之前，先导入 `xsys_msgs` 包中的 `setup.bash` 文件。

代码块

```
1  # x86
2  source ros2ws/install/setup.bash
3
4  # orin
5  source lyre_ros2/install/setup.bash
```

5.2 一键启动

5.2.1 一键启动功能

手动一键启动，启动之前确保自启动服务(proc_manager.service)已被disable及stop

(ps -ef | grep ros看看有没有ros相关的进程在，如果有，kill掉)：

手动启动进程管理器：

代码块

```
1  cd ros2ws
2  source install/setup.bash
3  ros2 launch proc_manager ty2.0_pro.launch.py
```

5.2.2 自启动功能启用/关闭

自启动功能默认被禁用，使用以下方法开启、关闭：

1. 使能自启动

代码块

```
1  sudo systemctl enable proc_manager.service
```

2. 禁用自启动

代码块

```
1  sudo systemctl disable proc_manager.service
```

5.2.3 服务启动配置说明

自启动服务配置如下，配置位于：

`/home/ubuntu/ros2ws/install/proc_manager/share/proc_manager/param/proc_manager_tg2.0_pro.json`，定义如下：

其中：

1. config：（定义全局配置参数）：

代码块

```
1  {
2      "config": {
3          "audio_player": "192.168.41.2",          # 定义语音播放
设备是在192.168.41.2 (Orin) 上
4          "audio_path": "/home/nvidia/data/speech/builtin",          # 定义
语音播放音频目录在/home/nvidia/data/speech
5          "audio_list": [ "0.mp3", "1.mp3", "2.mp3", "3.mp3", "4.mp3" ] # 定义遥控器控
制的语音文件列表
6      },
7      ...
8  }
9
```

2. proc：（定义服务进程）（开放给用户侧）：

代码块

```
1  {
2      ...
3      "proc": [
4          {
5              "name": "power_board",          # 服务名称
6              "enable": true,          # true: 生效,
false: 忽略
7              "shell": "ros2 launch power_board power_board.py",          # shell: 要执行的
shell命令
8              "boot_start": true,          # 开机是否自动启动,
true: 是, false: 否
9              "ready_topic": "power_board_state"          # 服务状态ready的
topic, 等待该topic信号标识服务已启动完成
10         },{
11             "name": "usb_sbus",
12             "enable": true,
13             "shell": "ros2 launch usb_sbus usb_sbus.launch.py",
14             "boot_start": true
```

```

15     },{
16         "name": "body_control",
17         "enable": true,
18         "shell": "ros2 launch body_control body_no_sbus.launch.py",
19         "boot_start": false,
20         "ready_topic": "bodycontrol_state",
21         "ready_timeout": 20
22     },{
23         "name": "motion",
24         "enable": true,
25         "shell": "ros2 launch motion_control motion.py"
26     }
27 ],
28 ...
29 }

```

5.3 IMU

5.3.1 获取IMU传感器信息格式1---标准格式

- 说明：获取IMU传感器数据信息，其中包含加速度、角速度、位姿四元数。
- 控制方式：topic
- 话题名称： `/imu`
- 数据定义位置： `sensor_msgs::msg::Imu`
- 数据格式：

代码块

```

1  std_msgs/Header header
2  geometry_msgs/Quaternion orientation
3  float64[9] orientation_covariance
4  geometry_msgs/Vector3 angular_velocity
5  float64[9] angular_velocity_covariance
6  geometry_msgs/Vector3 linear_acceleration
7  float64[9] linear_acceleration_covariance

```

5.3.2 获取IMU传感器信息格式2---自定义格式

- 说明：获取IMU传感器数据信息，其中包含加速度、角速度、位姿四元数、欧拉角和错误报警。
- 控制方式：topic
- 话题名称： `/imu/status`

- 数据定义位置: `bodyctrl_msgs::msg::Imu`
- 数据格式:

代码块

```
1 std_msgs/Header header
2 geometry_msgs/Quaternion orientation
3 geometry_msgs/Vector3 angular_velocity
4 geometry_msgs/Vector3 linear_acceleration
5 bodyctrl_msgs/Euler euler
6 uint32 error
7
8 float64[3] angular_velocity_covariance
9 float64[3] orientation_covariance
10 float64[3] linear_acceleration_covariance
```

error错误字段值定义如下:

代码块

```
1 error = 33072,    设备掉线
```

5.4 关节温度获取接口

- 说明: 所有关节温度获取, 以topic形式发布, 每秒1次;
- 数据定义位置: `bodyctrl_msgs::msg::MotorStatusMsg1`
- 数据格式:

代码块

```
1 std_msgs/Header header
2 MotorStatus1[] status
3
4 MotorStatus1定义如下:
5 uint16 name # MotorName
6 float32 motortemperature
7 float32 mostemperature
```

1. 获取头部关节温度信息

话题名称: `/head/motor_status`

2. 获取腰部关节温度信息

话题名称: `/waist/motor_status`

3. 获取双臂关节温度信息

话题名称: `/arm/motor_status`

4. 获取腿部关节温度信息

话题名称: `/leg/motor_status`

5.5 头部关节

5.5.1 状态获取接口

1. 获取头部关节信息

- 说明: 获取头部关节的状态信息, 其中包含关节的当前位置、速度、电流和温度。
- 控制方式: topic
- 话题名称: `/head/status`
- 数据定义位置: `bodyctrl_msgs::msg::MotorStatusMsg.msg`
- 数据格式:

代码块

```
1 std_msgs/Header header
2 MotorStatus[] status
3
4 MotorStatus定义如下:
5 uint16 name # MotorName
6 float32 pos # rad
7 float32 speed # rad
8 float32 current # A
9 float32 temperature # Mos温度
10 uint32 error
```

error错误字段值定义如下:

代码块

```
1 error = 33072, 设备掉线
2 error = 33073, 关节位置超限
3 error = 1, 关节电机过温
4 error = 2, 过流
5 error = 3, 电压过低
6 error = 4, 关节mos过温
7 error = 5, 堵转
8 error = 6, 电压过高
```

```
9 error = 7,  缺相
10 error = 8,  编码器错误
```

5.5.2 控制接口

1. 位置模式

- 说明：头部关节的位置控制接口，需要提供期望位置、期望速度、最大电流。
- 控制方式：topic
- 话题名称： `/head/cmd_pos`
- 数据定义位置： `bodyctrl_msgs::msg::CmdSetMotorPosition.msg`
- 数据格式：

代码块

```
1  std_msgs/Header header
2  SetMotorPosition[] cmds
3
4  # SetMotorPosition.msg
5  uint16 name # MotorName
6  float32 pos  # rad
7  float32 spd  # rad/s
8  float32 cur  # A
```

2. 力位混合模式

- 说明：头部关节的力位混合控制接口，需要提供期望位置、期望速度、前馈力矩、kp、kd系数。
- 控制方式：topic
- 话题名称： `/head/cmd_ctrl`
- 数据定义位置： `bodyctrl_msgs::msg::CmdMotorCtrl.msg`
- 数据格式：

代码块

```
1  std_msgs/Header header
2  MotorCtrl[] cmds
3
4  # MotorCtrl.msg
5  uint16 name
6  float32 kp
7  float32 kd
8  float32 pos
```

```
9 float32 spd
10 float32 tor
```

3. 速度模式

- 说明：关节的速度控制接口，需要提供期望速度、最大电流。
- 控制方式：topic
- 话题名称： `/head/cmd_vel`
- 数据定义位置： `bodyctrl_msgs::msg::CmdSetMotorSpeed.msg`
- 数据格式：

代码块

```
1 std_msgs/Header header
2 SetMotorSpeed[] cmds
3
4 #SetMotorSpeed.msg
5 uint16 name # MotorName
6 float32 spd # rad/s
7 float32 cur # A
```

4. 关节标零

- 说明：头部关节的标零接口。
- 控制方式：topic
- 话题名称： `/head/cmd_set_zero`
- 数据定义位置： `std_msgs::msg::String`
- 数据格式：

代码块

```
1 以字符串形式发送关节的id实现标零；
2 头部关节id：1,2,3；
```

5.6 腰部关节

5.6.1 状态获取接口

1. 获取腰部关节信息

- 说明：获取腰部关节的状态信息，其中包含关节的当前位置、速度、电流和温度。

- 控制方式: topic
- 话题名称: `/waist/status`
- 数据定义位置: `bodycltrl_msgs::msg::MotorStatusMsg.msg`
- 数据格式:

代码块

```
1 std_msgs/Header header
2 MotorStatus[] status
3
4 MotorStatus定义如下:
5 uint16 name      # MotorName
6 float32 pos      # rad
7 float32 speed    # rad
8 float32 current  # A
9 float32 temperature #
10 uint32 error
```

error错误字段值定义如下:

代码块

```
1 error = 33072,    设备掉线
2 error = 33073,    关节位置超限
3 error = 1,        关节电机过温
4 error = 2,        过流
5 error = 3,        电压过低
6 error = 4,        关节mos过温
7 error = 5,        堵转
8 error = 6,        电压过高
9 error = 7,        缺相
10 error = 8,       编码器错误
```

5.6.2 控制接口

1. 位置模式

- 说明: 关节的位置控制接口, 需要提供期望位置、期望速度、最大电流。
- 控制方式: topic
- 话题名称: `/waist/cmd_pos`
- 数据定义位置: `bodycltrl_msgs::msg::CmdSetMotorPosition.msg`
- 数据格式:

代码块

```
1  std_msgs/Header header
2  SetMotorPosition[] cmds
3
4  # SetMotorPosition.msg
5  uint16 name # MotorName
6  float32 pos # rad
7  float32 spd # rad/s
8  float32 cur # A
```

2. 力位混合模式

- 说明：关节的力位混合控制接口，需要提供期望位置、期望速度、前馈力矩、kp、kd系数。
- 控制方式：topic
- 话题名称： `/waist/cmd_ctrl`
- 数据定义位置： `bodyctrl_msgs::msg::CmdMotorCtrl.msg`
- 数据格式：

代码块

```
1  std_msgs/Header header
2  MotorCtrl[] cmds
3
4  # MotorCtrl.msg
5  uint16 name
6  float32 kp
7  float32 kd
8  float32 pos
9  float32 spd
10 float32 tor
```

3. 速度模式

- 说明：关节的速度控制接口，需要提供期望速度、最大电流。
- 控制方式：topic
- 话题名称： `/waist/cmd_vel`
- 数据定义位置： `bodyctrl_msgs::msg::CmdSetMotorSpeed.msg`
- 数据格式：

代码块

```

1 std_msgs/Header header
2 SetMotorSpeed[] cmds
3
4 #SetMotorSpeed.msg
5 uint16 name # MotorName
6 float32 spd # rad/s
7 float32 cur # A

```

4. 腰部关节标零

- 说明：腰部关节的标零接口。
- 控制方式：topic
- 话题名称： `/waist/cmd_set_zero`
- 数据定义位置： `std_msgs::msg::String`
- 数据格式：

代码块

```

1 以字符串形式发送关节的id实现标零；
2 腰部关节id: 31

```

5.7 双臂关节

5.7.1 状态获取接口

1. 获取双臂关节信息

- 说明：获取双臂关节的状态信息，其中包含关节的当前位置、速度、电流和温度。
- 控制方式：topic
- 话题名称： `/arm/status`
- 数据定义位置： `bodyctrl_msgs::msg::MotorStatusMsg.msg`
- 数据格式：

代码块

```

1 std_msgs/Header header
2 MotorStatus[] status
3
4 MotorStatus定义如下：
5 uint16 name # MotorName
6 float32 pos # rad
7 float32 speed # rad

```

```
8 float32 current # A
9 float32 temperature
10 uint32 error
```

error错误字段值定义如下：

代码块

```
1 error = 33072,    设备掉线
2 error = 33073,    关节位置超限
3 error = 1,        关节电机过温
4 error = 2,        过流
5 error = 3,        电压过低
6 error = 4,        关节mos过温
7 error = 5,        堵转
8 error = 6,        电压过高
9 error = 7,        缺相
10 error = 8,       编码器错误
```

5.7.2 控制接口

1. 位置模式

- 说明：关节的位置控制接口，需要提供期望位置、期望速度、最大电流。
- 控制方式：topic
- 话题名称： `/arm/cmd_pos`
- 数据定义位置： `bodyctrl_msgs::msg::CmdSetMotorPosition.msg`
- 数据格式：

代码块

```
1 std_msgs/Header header
2 SetMotorPosition[] cmds
3
4 # SetMotorPosition.msg
5 uint16 name # MotorName
6 float32 pos # rad
7 float32 spd # rad/s
8 float32 cur # A
```

2. 力位混合模式

- 说明：关节的力位混合控制接口，需要提供期望位置、期望速度、前馈力矩、kp、kd系数。

- 控制方式: topic
- 话题名称: `/arm/cmd_ctrl`
- 数据定义位置: `bodyctrl_msgs::msg::CmdMotorCtrl.msg`
- 数据格式:

代码块

```
1 std_msgs/Header header
2 MotorCtrl[] cmds
3
4 # MotorCtrl.msg
5 uint16 name
6 float32 kp
7 float32 kd
8 float32 pos
9 float32 spd
10 float32 tor
```

3. 速度模式

- 说明: 关节的速度控制接口, 需要提供期望速度、最大电流。
- 控制方式: topic
- 话题名称: `/arm/cmd_vel`
- 数据定义位置: `bodyctrl_msgs::msg::CmdSetMotorSpeed.msg`
- 数据格式:

代码块

```
1 std_msgs/Header header
2 SetMotorSpeed[] cmds
3
4 #SetMotorSpeed.msg
5 uint16 name # MotorName
6 float32 spd # rad/s
7 float32 cur # A
```

4. 电流模式

- 说明: 关节的电流控制接口, 需要提供期望电流。
- 控制方式: topic
- 话题名称: `/arm/cmd_current`

- 数据定义位置: `bodyctrl_msgs::msg::CmdSetMotorCurTor.msg`
- 数据格式:

代码块

```
1 std_msgs/Header header
2 SetMotorCurTor[] cmds
3
4 #SetMotorCurTor.msg
5 uint16 name # MotorName
6 float32 cur_tor
7 uint8 ctrl_status    #当前只支持电流模式
```

5. 手臂关节标零

- 说明: 双臂关节的标零接口。
- 控制方式: topic
- 话题名称: `/arm/cmd_set_zero`
- 数据定义位置: `std_msgs::msg::String`
- 数据格式:

代码块

```
1 以字符串形式发送关节的id实现标零;
2 双臂关节id:
3     左臂: 11---17,
4     右臂: 21---27;
```

5.8 双腿关节

5.8.1 状态获取接口

1. 获取双腿关节信息

- 说明: 获取双腿关节的状态信息, 其中包含关节的当前位置、速度、电流、温度和告警。
- 控制方式: topic
- 话题名称: `/leg/status`
- 数据定义位置: `bodyctrl_msgs::msg::MotorStatusMsg.msg`
- 数据格式:

代码块

```

1  std_msgs/Header header
2  MotorStatus[] status
3
4  MotorStatus定义如下:
5  uint16 name      # MotorName
6  float32 pos      # rad
7  float32 speed    # rad
8  float32 current  # A
9  float32 temperature
10 uint32 error

```

error错误字段值定义如下:

代码块

```

1  error = 33072,    设备掉线
2  error = 33073,    关节位置超限
3  error = 1,        关节电机过温
4  error = 2,        过流
5  error = 3,        电压过低
6  error = 4,        关节mos过温
7  error = 5,        堵转
8  error = 6,        电压过高
9  error = 7,        缺相
10 error = 8,        编码器错误

```

5.8.2 控制接口

1. 位置模式

- 说明: 关节的位置控制接口, 需要提供期望位置、期望速度、最大电流。
- 控制方式: topic
- 话题名称: `/leg/cmd_pos`
- 数据定义位置: `bodyctrl_msgs::msg::CmdSetMotorPosition.msg`
- 数据格式:

代码块

```

1  std_msgs/Header header
2  SetMotorPosition[] cmds
3
4  # SetMotorPosition.msg
5  uint16 name  # MotorName
6  float32 pos  # rad

```

```
7 float32 spd # rad/s
8 float32 cur # A
```

2. 力位混合模式

- 说明：关节的力位混合控制接口，需要提供期望位置、期望速度、前馈力矩、kp、kd系数。
- 控制方式：topic
- 话题名称： `/leg/cmd_ctrl`
- 数据定义位置： `bodyctrl_msgs::msg::CmdMotorCtrl.msg`
- 数据格式：

代码块

```
1 std_msgs/Header header
2 MotorCtrl[] cmds
3
4 # MotorCtrl.msg
5 uint16 name
6 float32 kp
7 float32 kd
8 float32 pos
9 float32 spd
10 float32 tor
```

3. 速度模式

- 说明：关节的速度控制接口，需要提供期望速度、最大电流。
- 控制方式：topic
- 话题名称： `/leg/cmd_vel`
- 数据定义位置： `bodyctrl_msgs::msg::CmdSetMotorSpeed.msg`
- 数据格式：

代码块

```
1 std_msgs/Header header
2 SetMotorSpeed[] cmds
3
4 #SetMotorSpeed.msg
5 uint16 name # MotorName
6 float32 spd # rad/s
7 float32 cur # A
```

4. 腿部关节标零

- 说明：双腿关节的标零接口。
- 控制方式：topic
- 话题名称： `/leg/cmd_set_zero`
- 数据定义位置： `std_msgs::msg::String`
- 数据格式：

代码块

```
1 以字符串形式发送关节的id实现标零；
2 双腿关节id：
3     左腿：51---56，
4     右腿：61---66；
```

5.9 手部关节

5.9.1 全局值说明

- 消息名称： `MotorName`
- 数据定义位置：
- 说明：手指各关节ID定义如下：

代码块

```
1 uint16 MOTOR_FINGER_LITTLE = 1,           //little finger 小指
2 uint16 MOTOR_FINGER_RING = 2,             //ring finger 无名指
3 uint16 MOTOR_FINGER_MIDDLE = 3,           //middle finger 中指
4 uint16 MOTOR_FINGER_FORE = 4,             //Fore finger 食指
5 uint16 MOTOR_FINGER_THUMB_BEND = 5,       //thumb bend 拇指弯曲
6 uint16 MOTOR_FINGER_THUMB_ROTATION = 6,   //thumb rotation 拇指旋转
```

5.9.2 状态获取接口

- 说明：获取手指各关节的状态信息，其中包含电机的当前位置、速度、电流。
- 控制方式：topic
- 话题名称： `/inspire_hand/state/left_hand`
和 `/inspire_hand/state/right_hand`
- 数据定义位置： `sensor_msgs::msg::JointState`
- 数据格式：

⚠ 注意：这里的数据（位置，速度，电流），指的是百分比

代码块

```
1 std_msgs/Header header
2 string[] name           #说明, id=1为小指, id=6为大拇指旋转
3 float64[] position      #百分比
4 float64[] velocity      #百分比
5 float64[] effort        #百分比
```

5.9.3 控制接口-topic

1. 控制手指各关节角度

- 说明：手部电机的位置控制接口，需要提供期望位置。
- 控制方式：topic
- 话题名称： `/inspire_hand/ctrl/left_hand` 和 `/inspire_hand/ctrl/right_hand`
- 数据定义位置： `sensor_msgs::msg::JointState.msg`
- 数据格式：

⚠ 注意：这里的position，指的仍然是手指张开的百分比；

代码块

```
1 std_msgs/Header header
2 string[] name
3 float64[] position      #百分比
```

5.9.4 控制接口-service

1. 控制手指各关节角度

- 说明：手指各个关节的位置控制接口，以百分比的方式。
- 控制方式：service
- 服务名称： `/inspire_hand/set_angle_flexible/left_hand` 和 `/inspire_hand/set_angle_flexible/right_hand`
- 数据定义位置： `bodyctrl_msgs::srv::set_angle_flexible.srv`
- 数据格式：

代码块

```
1 string[] name
2 float32[] angle_ratio
3 bool angle_accepted
```

5.9.5 控制手指各关节力矩

- 说明：手指各个关节的位置力矩，以百分比的方式。
- 控制方式：service
- 服务名称： `/inspire_hand/set_force/left_hand` 和 `/inspire_hand/set_force/right_hand`
- 数据定义位置： `bodyctrl_msgs::srv::set_force.srv`
- request:

代码块

```
1 float32 force0_Ratio #小拇指-百分比 (0代表0g, 1代表1000g)
2 float32 force1_Ratio #无名指-百分比
3 float32 force2_Ratio #中指-百分比
4 float32 force3_Ratio #食指-百分比
5 float32 force4_Ratio #大拇指弯曲-百分比
6 float32 force5_Ratio #大拇指旋转-百分比
```

- response:

代码块

```
1 bool force_accepted #是否设置成功
```

5.9.6 控制手指各关节速度

- 说明：手指各个关节的位置力矩，以百分比的方式。
- 控制方式：service
- 服务名称： `/inspire_hand/set_speed/left_hand` 和 `/inspire_hand/set_speed/right_hand`
- 数据定义位置： `bodyctrl_msgs::srv::set_speed.srv`
- request:

代码块

```
1 float32 speed0_Ratio #小拇指-百分比 (1代表800ms从最大角度到最小角度, 0.5代表1600ms, 0.25代表3200ms)
2 float32 speed1_Ratio #无名指-百分比
3 float32 speed2_Ratio #中指-百分比
4 float32 speed3_Ratio #食指-百分比
5 float32 speed4_Ratio #大拇指弯曲-百分比
```

```
6 float32 speed5_Ratio #大拇指旋转-百分比
```

- response:

代码块

```
1 bool speed_accepted #是否设置成功
```

5.9.7 清除手指各关节错误

- 说明：清除手指各个关节的错误接口。
- 控制方式：service
- 服务名称： `/inspire_hand/set_clear_error/left_hand` 和 `/inspire_hand/set_clear_error/right_hand`
- 数据定义位置： `bodyctrl_msgs::srv::set_clear_error.srv`
- request: `null`
- response:

代码块

```
1 bool setclear_error_accepted #是否设置成功
```

5.10 六维力

5.10.1 状态获取接口

1. 六维力信息

- 说明：获取六维力当前的信息。其中上报频率为100Hz。
- 控制方式：topic
- 话题名称： `/arm_6dof_left` 和 `/arm_6dof_right`
- 类型： `geometry_msgs::msg::WrenchStamped.msg`
- 数据格式：

代码块

```
1 Header header uint32 seq #序列号
2 Header header time stamp #时间戳
3 Header header string frame_id #帧id
4
```

- 5 Wrench wrench Vector3 force #3个方向的力
- 6 Wrench wrench Vector3 torque #3个方向的力矩

5.11 lyre 语音包

5.11.1 程序包及安装位置

lyre 语音包包括 lyre、lyre_msgs 和 lyre_examples 三个程序包。

lyre_ros2 是语音包的功能程序包，安装在 orin 主控中，安装位置为 /home/nvidia/ros2ws/lyre

- lyre_msgs 是语音包的接口定义。该接口定义同时安装在 x86 的 ros2ws 目录以及 orin ros2ws 目录。
- lyre_examples 是语音包接口的调用示例程序。该示例程序安装在 x86 的 ros2ws 目录。

lyre 语音包程序的目录结构如下：

代码块

```
1  |— ros2ws/                # x86
2  |   |— install
3  |   |   |— lyre_msgs      # 接口定义
4  |   |   |— README.md     # SDK 开发文档
5  |   |   |— lyre_examples  # 接口调用示例程序
6  |
7  |— ros2ws/                # orin
8  |   |— install
9  |   |   |— lyre_msgs      # 接口定义
10 |   |   |— README.md     # SDK 开发文档
11 |   |   |— lyre          # 功能包
```

5.11.2 语音交互功能

机器人提供有语音交互功能。设备开机以后，语音交互功能为关闭状态。通过遥控器按键可以开启语音交互功能。

5.11.2.1 遥控器开关

E、G、H 键在中间；F 键向上。

长按 A 键，可以打开/关闭语音交互功能。

5.11.2.2 语音对话方式

站立在机器人正前方 1.5 米左右的位置，正视机器人胸前麦克风上方的摄像头，与机器人进行对话。

5.11.3 启动语音包

语音包的 ROS 包名为 `lyre`，在 `systemd` 中的服务名称也是 `lyre`。

5.11.3.1 手动启动语音对话的 ROS 节点

在 `lyre_ros2` 程序包中使用下面的命令启动语音对话的 ROS 节点。

代码块

```
1 cd lyre_ros2
2 source install/setup.bash
3 ros2 launch lyre chat.launch.py
```

注意：机器人默认会在开始时自行启动 `lyre` 节点。同一时间只允许一个 `lyre` 进程运行。通过下面的命令检查系统中是否有其它 `lyre` 进程正在运行。

代码块

```
1 ps -ef | grep "ros2 launch lyre" | grep -v grep
```

如果有打印内容，则说明 `lyre` 进程正在运行。通过下面的命令可以关闭系统中正在运行的 `lyre` 进程。

代码块

```
1 pkill -INT -f "ros2 launch lyre"
```

注意：如果通过 `systemd` 配置了开机自启动服务，则需要在关闭自启动服务以后再结束进程。否则，手动结束的进程会被重新启动。具体操作参考后面的章节。

5.11.3.2 开机自启动服务

出厂时，语音包的 `systemd` 服务 `lyre` 会在开机时自行启动。

检查 `lyre` 服务是否会在开机时自启动

代码块

```
1 systemctl is-enabled lyre
```

如果显示 `enabled`，则说明 `lyre` 服务会在开机时自启动。

打开 `lyre` 服务开机自启动设置

代码块

```
1 sudo systemctl enable lyre
```

关闭 lyre 服务开机自启动设置

代码块

```
1 sudo systemctl disable lyre
```

5.11.4 启动模式

`lyre` 语音包提供了四种运行模式，分别是 `play`、`asr`、`audio`、`chat`。接入者可以根据实际情况选用其中的一种。

注意：启动模式是互斥的。同一时间，只可以允许 4 种启动模式中的一种。

- `play`：在此启动模式下，仅『音频播放』相关的功能可以使用。
- `asr`：在此启动模式下，仅『语音识别』相关的功能可以使用。
- `audio`：在此启动模式下，可以同时使用『语音识别』和『音频播放』功能，包括『文本播放』功能。
- `chat`：在此启动模式下，可以同时使用『语音识别』、『音频播放』和『语音对话』功能。

不同模式的节点的启动命令统一为：

代码块

```
1 ros2 launch lyre <mode>.launch.py
```

例如，启动 `audio` 模式的命令为：

代码块

```
1 ros2 launch lyre audio.launch.py
```

部署某个启动模式的开机自启动服务

`lyre_ros2` 程序包给出了自启动服务的部署脚本 `setup_systemd.sh`，该脚本位于 `lyre_ros2/scripts` 目录下。使用方法如下：

代码块

```
1 bash setup_systemd.sh <mode>[default:chat]
```

例如：将 `play` 启动模式部署为自启动服务的命令如下：

代码块

```
1 bash setup_systemd.sh play
```

5.11.5 接口示例程序

`lyre_examples` 程序包提供了部分 `CPP` 版本的接口调用示例。程序包的目录结构如下：

代码块

```
1 lyre_examples/  
2 |— bin/                # 接口调用示例的可执行程序  
3 |   |— lyre_example_play_file_srv  
4 |— cli/                # 接口调用示例的终端执行脚本  
5 |   |— call_play_file_srv.sh  
6 |   |— sub_play_progress_msg.sh  
7 |— src/                # 接口调用示例的 CPP 代码  
8 |   |— play_file_srv.cpp  
9 |— res/                # 静态资料  
10 |   |— test.mp3  
11 |— CMakeLists.txt      # 项目 CMakeLists.txt 示例  
12 |— package.xml         # 项目 ROS package.xml 示例
```

对于每个开放的语音接口，`lyre_examples` 包都提供有接口调用的示例代码、编译后的可执行程序、终端调用脚本，分别放置在 `src`、`bin`、`cli` 目录。

为了避免混淆，每个演示所用的可执行程序的文件名前面都加了 `lyre_example_` 作为前缀。每个用于订阅 Topic 的脚本的文件名都加了 `sub_` 作为前缀；每个用于发布 Topic 的脚本的文件名都加了 `pub_` 作为前缀。每个用于调用 Service 的脚本的文件名则都加了 `call_` 作为前缀。

对于二次开发的 CPP ROS2 项目，可以参考 `lyre_examples` 包中的方式导入 `lyre_msgs` 接口，再参考示例代码完成功能开发。

5.11.5.1 ROS package 配置

代码块

```
1 <package format="3">  
2   <depend>lyre_msgs</depend>  
3 </package>
```

5.11.5.2 项目 CMake 配置

```

1 find_package(lyre_msgs REQUIRED)
2
3ament_target_dependencies(
4    <your_target>
5    lyre_msgs
6 )

```

5.11.5.3 编译时、运行时依赖配置

x86 的 `ros2ws` 和 orin 的 `lyre_ros2` 提供了 `lyre_msgs` 包。在构建或者运行自己的 ROS 程序之前，需要先从这两个工作空间导入 `lyre_msgs` 依赖。

```

代码块
1 # x86
2 source ros2ws/install/setup.bash
3
4 # orin
5 source lyre_ros2/install/setup.bash

```

5.11.6 音频播放接口

5.11.6.1 播放本地文件（Service）

如果你已经明确知道 `lyre_ros2` 程序包所在的主控板中有一个音频文件，你可以使用下面的 ROS 服务播放这个文件。

```

代码块
1 # lyre_msgs/srv/PlayFile.srv
2 # 以本地文件的形式播放一段语音。
3
4 string SERVICE_NAME = /audio_play/play_file
5
6 string sid # Stream identifier (unique per audio stream).
7 uint32 seq # Sequence number (incremental per packet).
8 bool last # Last flag (true if this is the final packet).
9 bool force # Force playback (stop all running tasks and play immediately).
10
11 string path # Absolute path to the audio file in the local file system.
12 ---
13
14 int8 CODE_OK = 0
15 int8 CODE_INVALID_PARAMS = 1
16 int8 CODE_FAILED = -1
17

```

```

18  string sid      # Playback stream ID (generated internally if absent).
19  int8   code     # Status code.
20  string message  # Human-readable status message.

```

接口调用的 C++ 示例代码请参考 `lyre_examples` 程序包中的 `play_text_srv`。

5.11.6.2 播放远程文件 (Service)

你还可以使用下面的 ROS 服务来播放一个来自网络的音频文件。

代码块

```

1  # lyre_msgs/srv/PlayUrl.srv
2  # 以文件 URL 的形式播放一段语音。
3
4  string SERVICE_NAME = /audio_play/play_url
5
6  string sid      # Stream identifier (unique per audio stream).
7  uint32 seq      # Sequence number (incremental per packet).
8  bool   last     # Last flag (true if this is the final packet).
9  bool   force    # Force playback (stop all running tasks and play immediately).
10
11 string url      # URL to the audio file in the network.
12 ---
13
14 int8 CODE_OK = 0
15 int8 CODE_INVALID_PARAMS = 1
16 int8 CODE_FAILED = -1
17
18 string sid      # Playback stream ID (generated internally if absent).
19 int8   code     # Status code.
20 string message  # Human-readable status message.

```

接口调用的 C++ 示例代码请参考 `lyre_examples` 程序包中的 `play_url_srv`。

5.11.6.3 播放文本 (Service)

如果你需要从别的主控板访问音频播放服务，可以使用下面的 ROS 服务来播放一个字符串。

代码块

```

1  # lyre_msgs/srv/PlayText.srv
2  # 以文本的形式播放一段语音。
3
4  string SERVICE_NAME = /audio_play/play_text
5
6  string sid      # Stream identifier (unique per audio stream).

```

```

7  uint32 seq    # Sequence number (incremental per packet).
8  bool  last    # Last flag (true if this is the final packet).
9  bool  force   # Force playback (stop all running tasks and play immediately).
10
11  string text   # Text to be synthesized into speech.
12
13  string token  # System api (unavailable for applications).
14  string output # System api (unavailable for applications).
15  ---
16
17  int8 CODE_OK = 0
18  int8 CODE_INVALID_PARAMS = 1
19  int8 CODE_FAILED = -1
20
21  string sid    # Playback stream ID (generated internally if absent).
22  int8  code    # Status code.
23  string message # Human-readable status message.

```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `play_text_srv`。

注意：此功能仅在 `audio` 和 `chat` 启动模式下才可用。

5.11.6.4 监听播放事件 (Topic)

播放音频的过程中会通过下面的 Topic 来发布播放事件：

代码块

```

1  # lyre_msgs/msg/PlayEvent.msg
2  # Play 音频播放的事件。
3
4  string TOPIC_NAME = /audio_play/event
5
6  int8 EVENT_STARTED    = 0
7  int8 EVENT_COMPLETED = 1
8  int8 EVENT_STOPPED   = 2
9  int8 EVENT_CANCELLED = 3
10 int8 EVENT_FAILED    = 4
11
12 string sid
13 uint32 seq
14 int8  event
15 string message

```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `play_event_msg`。

5.11.6.5 监听播放进度 (Topic)

播放音频的过程中会通过下面的 Topic 来发布播放的进度及总时长（单位：秒）：

代码块

```
1 # lyre_msgs/msg/PlayProgress.msg
2 # Play 音频播放的进度。
3
4 string TOPIC_NAME = /audio_play/progress
5
6 string sid
7 uint32 seq
8 float64 position
9 float64 duration
```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `play_progress_msg`。

5.11.6.6 停止音频播放 (Service)

通过下面的 ROS 服务来停止音频播放（停止以后无法恢复）：

代码块

```
1 # lyre_msgs/srv/PlayStop.srv
2 # 停止当前的音频播放任务（不可恢复）。
3
4 string SERVICE_NAME = /audio_play/stop
5
6 ---
```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `play_stop_srv`。

5.11.6.7 暂停音频播放 (Service)

通过下面的 ROS 服务来停止音频播放（暂停以后可以调用 Resume 服务恢复播放）：

代码块

```
1 # lyre_msgs/srv/PlayPause.srv
2 # 暂停当前的音频播放任务。
3
4 string SERVICE_NAME = /audio_play/pause
5
6 ---
```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `play_pause_srv`。

5.11.6.8 恢复音频播放 (Service)

通过下面的 ROS 服务来恢复播放暂停中的音频：

代码块

```
1 # lyre_msgs/srv/PlayResume.srv
2 # 恢复已暂停的音频播放任务。
3
4 string SERVICE_NAME = /audio_play/resume
5
6 ---
```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `play_resume_srv`。

5.11.7 语音识别接口

设备集成有语音识别模块，可以提供 ASR 语音识别功能。

5.11.7.1 监听唤醒事件 (Topic)

在设备周围喊出关键词，可以触发唤醒事件。不同设备的唤醒关键词请参考用户手册。

在代码中可以通过以下 ROS 话题来监听唤醒消息：

代码块

```
1 # lyre_msgs/msg/AsrKeyword.msg
2 # ASR 唤醒时识别到的关键字。
3
4 string TOPIC_NAME = /audio_asr/keyword
5
6 string keyword
7 int32 angle
```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `asr_keyword_msg`。

5.11.7.2 监听语音识别结果 (Topic)

设备可以将周围的声音转化为文本。可以通过以下 ROS 话题获取识别到的文本：

代码块

```
1 # lyre_msgs/msg/AsrIat.msg
2 # ASR 语音识别到的内容。
3
```

```

4  string TOPIC_NAME = /audio_asr/iat
5
6  string id
7  string text

```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `asr_iat_msg`。

5.11.7.3 监听其它事件 (Topic)

设备还会发布其它 ASR 相关的事件，事件类型定义在下面的 ROS 话题中：

代码块

```

1  # lyre_msgs/msg/AsrEvent.msg
2  # ASR 设备报出的事件。
3
4  string TOPIC_NAME = /audio_asr/event
5
6  int8 EVENT_ERROR    = 2  # 出错事件。arg1 是错误码。
7  int8 EVENT_STATE    = 3  # 服务状态事件。
8  int8 EVENT_WAKEUP   = 4  # 唤醒事件。
9  int8 EVENT_SLEEP    = 5  # 休眠事件。
10
11  int8 EVENT_PRE_SLEEP      = 10 # 准备休眠事件。
12  int8 EVENT_CONNECTED_TO_SERVER = 13 # 与服务端建立连接。
13  int8 EVENT_SERVER_DISCONNECTED = 14 # 与服务端断开连接。
14
15  int8 event
16  int8 arg1

```

接口调用的 CPP 示例代码请参考 `lyre_examples` 程序包中的 `asr_event_msg`。

5.11.7.4 aiui_msg 事件

SDK 兼容旧版本的 `/xunfei/aiui_msg` 话题，这个话题已经被标记为 Deprecated，未来很可能会被移除。

代码块

```

1  string TOPIC_NAME = /xunfei/aiui_msg

```

该话题的类型是 `std_msgs::msg::String`，可以通过下面的示例脚本获取到：

代码块

```
1 ros2 topic echo -f /xunfei/aiui_msg
```

5.11.8 语音对话接口

5.11.8.1 语音交互开关

`lyre` 节点接收一个 ROS 话题来控制语音交互的可用状态。

代码块

```
1 string TOPIC_NAME = /audio_chat/enable
```

话题的类型是 `std_msgs/msg/Bool`。

对话开启的示例脚本：

代码块

```
1 ros2 topic pub -1 /audio_chat/enable std_msgs/msg/Bool 'data: true'
```

对话关闭的示例脚本：

代码块

```
1 ros2 topic pub -1 /audio_chat/enable std_msgs/msg/Bool 'data: false'
```

5.12 深度相机

启动奥比相机方法-ROS2

大模型Orin-连接头部相机（192.168.41.2）：

代码块

```
1 sudo systemctl start orbbec_head.service
```

导航Orin-连接腰部和后背相机（192.168.41.3）：

代码块

```
1 sudo systemctl start orbbec_waist.service
2 sudo systemctl start orbbec_back.service
```

(注：请勿在x86系统订阅相机数据，这会占用大量通信带宽，严重时会影响ros2通信从而导致机器人失控)

相机话题列表：

代码块

```
1  #头部相机
2  /ob_camera_head/color/camera_info      #彩色相机内参
3  /ob_camera_head/color/image_raw        #彩色原始图像（默认 BGR8 格式，分辨率如
    1280×720）
4  /ob_camera_head/color/metadata          #彩色图像元数据
5  /ob_camera_head/depth/camera_info      #深度相机内参
6  /ob_camera_head/depth/image_raw        #深度原始图像
7  /ob_camera_head/depth/metadata          #深度图像元数据
8  /ob_camera_head/depth_filter_status    #深度滤波状态
9  /ob_camera_head/depth_to_color          #深度相机→彩色相机的坐标变换
10 /ob_camera_head/depth/points            #点云
11 #腰部相机
12 /ob_camera_waist/color/camera_info      #彩色相机内参
13 /ob_camera_waist/color/image_raw        #彩色原始图像（默认 BGR8 格式，分辨率如
    1280×720）
14 /ob_camera_waist/color/metadata          #彩色图像元数据
15 /ob_camera_waist/depth/camera_info      #深度相机内参
16 /ob_camera_waist/depth/image_raw        #深度原始图像
17 /ob_camera_waist/depth/metadata          #深度图像元数据
18 /ob_camera_waist/depth_filter_status    #深度滤波状态
19 /ob_camera_waist/depth_to_color          #深度相机→彩色相机的坐标变换
20 /ob_camera_waist/depth/points            #点云
21 #后背相机
22 /ob_camera_back/color/camera_info        #彩色相机内参
23 /ob_camera_back/color/image_raw          #彩色原始图像（默认 BGR8 格式，分辨率如
    1280×720）
24 /ob_camera_back/color/metadata           #彩色图像元数据
25 /ob_camera_back/depth/camera_info        #深度相机内参
26 /ob_camera_back/depth/image_raw          #深度原始图像
27 /ob_camera_back/depth/metadata           #深度图像元数据
28 /ob_camera_back/depth_filter_status      #深度滤波状态
29 /ob_camera_back/depth_to_color           #深度相机→彩色相机的坐标变换
30 /ob_camera_back/depth/points             #点云
```

5.13 电池状态及电量获取

1. 电池状态

- 说明：获取电池状态信息。其中上报频率为1Hz。

- 充电状态：master_battery_current字段，负值表示在放电，正值表示在充电
- 控制方式：topic
- 话题名称：/power/battery/status
- 类型：`bodyctrl_msgs::msg::PowerBatteryStatus.msg`
- 数据格式：

代码块

```

1  # @brief 电源板电压、电流、电量、温度等信息
2
3  std_msgs/Header header
4
5  # battery status information
6  int32 battery_installed # 0x00都不存在, 0x01小电池存在, 0x02大电池存在, 0x03大小电
   池都存在
7  int32 battery_working   # 0x10大电池, 0x01小电池
8
9  float32 master_battery_voltage # 电池电压信息
10 float32 master_battery_current # 电池电流信息, 负值表示在放电, 正值表示在充电
11 float32 master_battery_power   # 电池电量
12
13 float32 little_battery_voltage # 小电池电压信息
14 float32 little_battery_current # 小电池电流信息, 负值表示在放电, 正值表示在充电
15 float32 little_battery_power   # 小电池电量
16
17 int8 pg12a # 00低, 01高
18 int8 pg12b # 00低, 01高
19 int8 pg12c # 00低, 01高
20 int8 pg12d # 00低, 01高
21 int8 pg5cd # 00低, 01高
22 int8 pg5ab # 00低, 01高
23
24 int8 pgrdc2   # 00低, 01高
25 int8 pgrdc1   # 00低, 01高
26 int8 pgheader # 00低, 01高
27 int8 pgbutton2 # 00低, 01高
28

```

2. 电源板状态

- 说明：获取电源板状态信息。其中上报频率为1Hz。
- 控制方式：topic
- 话题名称：/power/board/status

- 类型: bodyctrl_msgs::msg::PowerStatus.msg

- 数据:

代码块

```
1  # @brief 电源板电压、电流、电量、温度等信息
2
3  std_msgs/Header header
4
5  # temperature information
6  float32 waist_temp # 腰部MOS温度
7  float32 arm_a_temp # 臂A MOS温度
8  float32 arm_b_temp # 臂B MOS温度
9  float32 leg_a_temp # 腿A MOS温度
10 float32 leg_b_temp # 腿B MOS温度
11
12 float32 waist_temp_max # 腰部MOS温度最大值
13 float32 arm_a_temp_max # 臂A MOS温度最大值
14 float32 arm_b_temp_max # 臂B MOS温度最大值
15 float32 leg_a_temp_max # 腿A MOS温度最大值
16 float32 leg_b_temp_max # 腿B MOS温度最大值
17
18 float32 waist_temp_min # 腰部MOS温度最小值
19 float32 arm_a_temp_min # 臂A MOS温度最小值
20 float32 arm_b_temp_min # 臂B MOS温度最小值
21 float32 leg_a_temp_min # 腿A MOS温度最小值
22 float32 leg_b_temp_min # 腿B MOS温度最小值
23
24 # current information
25 float32 arm_a_curr # 臂A电流
26 float32 arm_b_curr # 臂B电流
27 float32 leg_a_curr # 腿A电流
28 float32 leg_b_curr # 腿B电流
29 float32 waist_curr # 腰部电流
30 float32 head_curr # 头部电流
31
32 float32 arm_a_curr_max # 臂A电流最大值
33 float32 arm_b_curr_max # 臂B电流最大值
34 float32 leg_a_curr_max # 腿A电流最大值
35 float32 leg_b_curr_max # 腿B电流最大值
36 float32 waist_curr_max # 腰部电流最大值
37 float32 head_curr_max # 头部电流最大值
38
39 float32 arm_a_curr_min # 臂A电流最小值
40 float32 arm_b_curr_min # 臂B电流最小值
41 float32 leg_a_curr_min # 腿A电流最小值
42 float32 leg_b_curr_min # 腿B电流最小值
```

```

43 float32 waist_curr_min # 腰部电流最小值
44 float32 head_curr_min # 头部电流最小值
45
46 float32 arm_a_volt # 臂A电压
47 float32 arm_b_volt # 臂B电压
48 float32 leg_a_volt # 腿A电压
49 float32 leg_b_volt # 腿B电压
50 float32 waist_volt # 腰部电压
51 float32 bus_volt # 母线电压
52
53 float32 arm_a_volt_max # 臂A电压最大值
54 float32 arm_b_volt_max # 臂B电压最大值
55 float32 leg_a_volt_max # 腿A电压最大值
56 float32 leg_b_volt_max # 腿B电压最大值
57 float32 waist_volt_max # 腰部电压最大值
58 float32 bus_volt_max # 母线电压最大值
59
60 float32 arm_a_volt_min # 臂A电压最小值
61 float32 arm_b_volt_min # 臂B电压最小值
62 float32 leg_a_volt_min # 腿A电压最小值
63 float32 leg_b_volt_min # 腿B电压最小值
64 float32 waist_volt_min # 腰部电压最小值
65 float32 bus_volt_min # 母线电压最小值
66
67 # version information
68 string software_version # 软件版本
69 string hardware_version # 硬件版本
70
71 # battery status information
72 float32 battery_voltage # 电池电压信息
73 float32 battery_current # 电池电流信息
74 float32 battery_power # 电池电量
75

```

3. 急停按键状态

- 说明：获取急停按键状态信息。其中上报频率为1Hz。
- 控制方式：topic
- 话题名称：/power/board/key_status
- 类型：`bodyctrl_msgs::msg::PowerBoardKeyStatus.msg`
- 数据格式：

代码块

```
1 # @brief 电源板按键状态
```

```

2
3 std_msgs/Header header
4
5 # power board status
6 uint32 work_time          # 工作时间
7 std_msgs/Bool is_estop    # 急停是否被按下
8 std_msgs/Bool is_remote_estop # 软急停是否被按下
9 std_msgs/Bool is_power_on  # 电源是否正常供电
10

```

5.14 遥控器接口

1. 遥控器事件数据

- 说明：获取遥控器原始数据
- 控制方式：topic
- 话题名称：/sbus_data/event
- 类型：`bodyctrl_msgs::msg::SbusData.msg`
- 数据格式：

代码块

```

1  int32 KEY_NONE=0
2  int32 KEY_A_UP=1      # A键抬起事件
3  int32 KEY_A_DOWN=2    # A键按下事件
4  int32 KEY_B_UP=3      # B键抬起事件
5  int32 KEY_B_DOWN=4    # B键按下事件
6  int32 KEY_C_UP=5      # C键抬起事件
7  int32 KEY_C_DOWN=6    # C键按下事件
8  int32 KEY_D_UP=7      # D键抬起事件
9  int32 KEY_D_DOWN=8    # D键按下事件
10 int32 KEY_E_UP=9       # E键上拨事件
11 int32 KEY_E_MID=10     # E键回中事件
12 int32 KEY_E_DOWN=11    # E键下拨事件
13 int32 KEY_F_UP=12      # F键上拨事件
14 int32 KEY_F_MID=13     # F键回中事件
15 int32 KEY_F_DOWN=14    # F键下拨事件
16 int32 KEY_G_LEFT=15    # G键左拨事件
17 int32 KEY_G_MID=16     # G键回中事件
18 int32 KEY_G_RIGHT=17   # G键右拨事件
19 int32 KEY_H_LEFT=18    # H键左拨事件
20 int32 KEY_H_MID=19     # H键回中事件
21 int32 KEY_H_RIGHT=20   # H键右拨事件
22
23 std_msgs/Header header

```

```

24
25  int32 key_event_new      # 按键事件新值
26  int32 key_event_old     # 按键事件旧值
27                               # 例如 G键从右边拨到中间位置，则key_event_old=KEY_G_RIGHT，
key_event_new=KEY_G_MID
28
29  int8 button_a           # A键键值，-1：松开，1：按下
30  int8 button_b           # B键键值，-1：松开，1：按下
31  int8 button_c           # C键键值，-1：松开，1：按下
32  int8 button_d           # D键键值，-1：松开，1：按下
33  int8 button_e           # E键键值，<-0.5：上拨，-0.5~0.5：中间，>0.5：下拨
34  int8 button_f           # F键键值，<-0.5：上拨，-0.5~0.5：中间，>0.5：下拨
35  int8 button_g           # G键键值，<-0.5：左拨，-0.5~0.5：中间，>0.5：右拨
36  int8 button_h           # H键键值，<-0.5：右拨，-0.5~0.5：中间，>0.5：左拨
37
38  float32 x1              # 左摇杆X方向值（左右方向），范围-1.0~1.0
39  float32 y1              # 左摇杆Y方向值（上下方向），范围-1.0~1.0
40  float32 x2              # 右摇杆X方向值（左右方向），范围-1.0~1.0
41  float32 y2              # 右摇杆Y方向值（上下方向），范围-1.0~1.0

```

5.15 软件版本信息获取

完整版本信息文件（包含x86、orin、firmware等完整版本信息描述文件）定义在本地软件包的安装目录中：

ros2路径是：/home/ubuntu/ros2ws/version_info.json

格式如下：

代码块

```

1  {
2    "product": "TG2.0-XXX",
3    "version": "vX.X.X",
4    "commit": "bu/integration/tiangong2.0/XXX,xxx",
5    "release_time": "xxxx-xx-xx xx:xx:xx",
6    "x86": [
7      {
8        "module": "benti",
9        "status": "enable",
10       "product": "TG2.0-XXX",
11       "platform": "linux-x86",
12       "version": "vX.X.X",
13       "file_name": "TG2.0-Pro_linux-x86_general_vX.X.X.tar.gz",
14       "commit": "bu/project/tiangong2.0/pro",
15       "install_dir_name": "ros2ws",
16       "install_address": "192.168.41.1",

```

```

17     "release_time": "XXXX-XX-XX XX:XX:XX"
18 },
19 {
20     "module": "ota",
21     "status": "disable",
22     "product": "ota",
23     "platform": "linux-x86",
24     "version": "vX.X.X",
25     "file_name": "ota_linux-x86_general_vX.X.X.tar.gz",
26     "commit": "bu/base/ota",
27     "install_dir_name": "ota",
28     "install_address": "192.168.41.1",
29     "release_time": "XXXX-XX-XX XX:XX:XX"
30 }
31 ]
32 ...
33 }

```

5.16 bag录制接口

直接使用launch文件可以启动ros2 topic录制，并存储到默认路径（/home/ubuntu/bags）。录制按照100M bag分片，录制目录最大存储容量为4G，超过限制后会自动删除最早文件夹以及bag文件。

代码块

```

1  #运行指定topic录制，录制列表存储在
   /home/ubuntu/ros2ws/install/utils/lib/utils/bag_record/config/record.json
   /topic_list，默认列表中存储了本体控制的关键topic
2  ros2 launch utils record_trigger.py

```

5.17 蓝牙模块

机器人支持通过蓝牙实现设备控制与状态监控等，具体内容详见以下文档链接：

[📖 诊断及蓝牙接入接口](#)

5.18 传统方法控制接口

手柄向上拨动E键，可通过下述ROS service切换状态，通过topic下发速度指令，遥控器按键和摇杆无效（只有C有效），E置中恢复遥控器功能

5.18.1 运控模式切换

- 接口名称：/hric/motion/set_motion_mode
- 类型：ROS service

- **描述：** 请求切换运动模式
- **发布模块：** 运动控制
- **client模块：** 机器人管理
- **消息类型：** hric_msgs/SetMotionMode.srv

Service msg	参数名	参数类型	是否必填	备注
请求字段	walk_mode_request	uint8	是	#请求切换的运动模式 0: start 1: stop 2: zero 3: stand 4: walk
	is_need_swing_arm	bool	是	是否需要控制手臂 true：需要 false：不需要
回复字段	sucess	bool	是	表示服务调用是否成功
	error_code	uint32	是	错误码 NONE=0 UNKNOWN=400

• **发布示例：**

代码块

```
1 cd ros_ws/  
2 source install_isolated/setup.bash  
3 roscall /hric/motion/set_motion_mode "{  
4   walk_mode_request: 2,  
5   is_need_swing_arm: false  
6 }" #####机器人进入回零状态。
```

代码块

```

1 cd ros2ws/
2 source install/setup.bash
3 ###机器人进入回零状态。
4 ros2 service call /hric/motion/set_motion_mode hric_msgs/srv/SetMotionMode "
  {walk_mode_request: 2, is_need_swing_arm: false}"

```

5.18.2 矢量速度控制接口

- 接口名称：/hric/robot/cmd_vel
- 类型：topic
- 描述：发布速度指令，控制机器人运动。
- 发布模块：遥控器
- 订阅模块：机器人管理
- 消息类型：geometry_msgs/TwistStamped.msg
- 消息频率：20hz
- 发布示例：

代码块

```

1 cd ros2ws/
2 source install_isolated/setup.bash
3 rostopic pub /hric/robot/cmd_vel geometry_msgs/TwistStamped "{
4   header: {
5     seq: 0,
6     stamp: {secs: 0, nsecs: 0},
7     frame_id: 'base_link'
8   },
9   twist: {
10    linear: {x: 0.5, y: 0.0, z: 0.0},
11    angular: {x: 0.0, y: 0.0, z: 0.0}
12  }
13 }"

```

代码块

```

1 cd ros2ws/
2 source install/setup.bash
3 ros2 topic pub -r 10 /hric/robot/cmd_vel geometry_msgs/msg/Twist "{
4   linear:
5     x: 0.5
6     y: 0.0
7     z: 0.0

```

```
8     angular:
9       x: 0.0
10      y: 0.0
11      z: 0.0
12  }"
```

5.18.3 机器人下肢运动状态发布

- 接口名称：/hric/motion/status
- 类型：topic
- 描述：发布运动控制模块状态和速度
- 发布模块：运动控制
- 发布频率：20hz
- 订阅模块：机器人管理、定位、导航
- 消息类型：hric_msgs/MotionStatus.msg

Topic msgs	参数名	参数 类型	是否 必填	备注
	header	Header	是	时间戳和坐标系
	velocity	geometry_msgs/Twist.msg	是	机器人腰部坐标系速度
	walk_mode	uint8	是	行走状态 0: stop 1: zero 2: zero_2_stand 3: stand 4: stand_2_walk 5: walk 6: start
	is_console_control	bool	是	是否为遥控器操控 true: 导航操控 false: 遥控器操纵

is_swing_arm	bool	是	是否为控制手臂状态 true: 摆臂状态 false: 非摆臂状态
error_code	uint32	是	错误码 NONE=0 UNKNOWN=400

● 消息格式示例：

代码块

```
1  {
2    "header": {
3      "seq": 1,
4      "stamp": {
5        "secs": 1622548800,
6        "nsecs": 0
7      },
8      "frame_id": ""
9    },
10   "velocity": {
11     "linear": {
12       "x": 1.0,
13       "y": 0.0,
14       "z": 0.0
15     },
16     "angular": {
17       "x": 0.0,
18       "y": 0.0,
19       "z": 0.0
20     }
21   },
22   "walk_mode": 6,
23   "error_code": 0
24 }
```

5.18.4 机器人姿态控制接口

适用于motion_control，E往上拨的情况下，通过下述话题控制浮动基高度和姿态

- 接口名称： /hric/robot/float_base_rpyz_cmd
- 类型： topic
- 描述： 站立状态下发布腰部rpyz指令，控制机器人姿态。

- **订阅模块：** 机器人管理
- **消息类型：** hric_msgs/FloatBaseRPYZ.msg
- **消息频率：** 400hz
- **发布示例：**

代码块

```
1 cd ros_ws/
2 source install_isolated/setup.bash
3 rostopic pub /hric/robot/float_base_rpyz_cmd hric_msgs/FloatBaseRPYZ "{
4   roll: 0.0, pitch: 0.05, yaw: 0, z: 0.88
5 }"
```

代码块

```
1 cd ros2_ws/
2 source install/setup.bash
3 ros2 topic pub /hric/robot/float_base_rpyz_cmd hric_msgs/msg/FloatBaseRPYZ "{
4   roll: 0.0, pitch: 0.05, yaw: 0, z: 0.88
5 }"
```

pro版：

z[0.78,0.88]m 默认0.88m

推荐在默认高度0.88m时，耦合情况下运动范围：

代码块

```
1 yaw[-0.8,0.8]rad
2 pitch[-0.05,0.4]rad
3 roll[-0.15,0.15]rad
```

运控内部对指令进行了速度限制：

代码块

```
1 yaw 0.4rad/s
2 pitch 0.4rad/s
3 roll 0.4rad/s
4 z 0.2m/s
```

5.18.5 机器人当前姿态状态反馈

适用于motion_control_joint，当机器人处于站立状态时，获取浮动基座的实际rpyz。

- 接口名称： /hric/robot/float_base_rpyz_status
- 类型： topic
- 描述： 发布站立状态下机器人浮动基座的实际rpyz
- 发布模块： 运动控制
- 发布频率： 400hz
- 订阅模块： 机器人管理、操作
- 消息类型： hric_msgs/FloatBaseRPYZ.msg
- 消息格式示例：

代码块

```
1  ---
2  roll: 4.081035528590296e-05
3  pitch: 0.02553808313186995
4  yaw: -0.0002636909484863281
5  z: 0.8828393155853616
```

5.19 强化学习控制接口

手柄向上拨动E键，可通过下述ROS service切换状态，通过topic下发速度指令，遥控器按键和摇杆无效（只有C有效），E置中恢复遥控器功能。

5.19.1 运控模式切换

- 接口名称： /hric/motion/set_motion_mode
- 类型： ROS service
- 描述： 请求切换运动模式
- 发布模块： 运动控制
- client模块： 机器人管理
- 消息类型： hric_msgs/SetMotionMode.srv

Service msg	参数名	参数类型	是否必填	备注
		uint8	是	#请求切换的运动模式

请求字段	walk_mode_request			1: stop 2: zero 3: stand 4: walk 5: run
	is_need_swing_arm	bool	是	walk是否需要摆臂 true: 需要 false: 不需要 (当前版本RL还不支持设置该功能，因此设置true或false都可以)
回复字段	sucess	bool	是	表示服务调用是否成功
	error_code	uint32	是	错误码 NONE=0 UNKNOWN=400

发布示例：

代码块

```
1 cd ros_ws/  
2 source install_isolated/setup.bash  
3 rosservice call /hric/motion/set_motion_mode "{  
4   walk_mode_request: 2,  
5   is_need_swing_arm: false  
6 }" #####机器人进入回零状态。
```

代码块

```
1 cd ros2_ws/  
2 source install/setup.bash  
3 #####机器人进入回零状态。  
4 ros2 service call /hric/motion/set_motion_mode hric_msgs/srv/SetMotionMode "  
   {walk_mode_request: 2, is_need_swing_arm: false}"
```

5.19.2 动作编号调用接口

- 接口名称： /hric/motion/set_motion_number
- 类型： ROS service

- **描述：** 请求调用动作
- **发布模块：** 运动控制
- **订阅模块：** 机器人管理
- **消息类型：** hric_msgs/SetMotionNumber.srv

Service msg	参数名	参数类型	是否必填	备注
请求字段	is_motion	bool	是	仅站立+E上拨状态下有效，置为true立即触发动作，false不会打断动作，true状态下会一直循环当前动作
	motion_number	int32	是	1~5，1为招手
回复字段	sucess	bool	是	表示服务调用是否成功

• **发布示例：**

代码块

```
1 cd ros_ws/  
2 source install_isolated/setup.bash  
3 rosservice call /hric/motion/set_motion_number "{  
4   is_motion: true,  
5   motion_number: 0  
6 }" #####机器人执行招手动作
```

代码块

```
1 cd ros2_ws/  
2 source install/setup.bash  
3 #####机器人执行招手动作  
4 ros2 service call /hric/motion/set_motion_number hric_msgs/srv/SetMotionNumber  
   "{is_motion: true, motion_number: 0}"
```

5.19.3 矢量速度控制接口

- 接口名称: /hric/robot/cmd_vel
- 类型: topic
- 描述: 发布速度指令, 控制机器人运动。
- 发布模块: 遥控器
- 订阅模块: 机器人管理
- 消息类型: geometry_msgs/TwistStamped.msg
- 消息频率: 20hz
- 发布示例:

代码块

```
1 cd ros_ws/  
2 source install_isolated/setup.bash  
3 rostopic pub /hric/robot/cmd_vel geometry_msgs/TwistStamped "{  
4   header: {  
5     seq: 0,  
6     stamp: {secs: 0, nsecs: 0},  
7     frame_id: 'base_link'  
8   },  
9   twist: {  
10    linear: {x: 0.5, y: 0.0, z: 0.0},  
11    angular: {x: 0.0, y: 0.0, z: 0.0}  
12  }  
13 }"
```

代码块

```
1 ros2 topic pub /hric/robot/cmd_vel geometry_msgs/msg/TwistStamped "header:  
2   stamp:  
3     sec: 0  
4     nanosec: 0  
5     frame_id: 'base_link'  
6   twist:  
7     linear:  
8       x: 1.0  
9       y: 0.0  
10      z: 0.0  
11     angular:  
12       x: 0.0  
13       y: 0.0"
```

5.19.4 机器人下肢运动状态发布

- 接口名称: /hric/motion/status
- 类型: topic
- 描述: 发布运动控制模块状态和速度
- 发布模块: 运动控制
- 发布频率: 20hz
- 订阅模块: 机器人管理、定位、导航
- 消息类型: hric_msgs/MotionStatus.msg

Topic msgs	参数名	参数 类型	是否 必填	备注
	header	Header	是	时间戳和坐标系
	velocity	geometry_msgs/Twist.msg	是	机器人腰部坐标系速度
	walk_mode	uint8	是	行走状态 0: stop, 1: zero, 3: stand, 5: walk, 7: run
	is_console_control	bool	是	是否为遥控器操控 true: 导航操控 false: 遥控器操纵
	is_swing_arm	bool	是	是否为walk摆臂状态（暂无该功能）
	error_code	uint32	是	错误码 NONE=0 UNKNOWN=400

- 消息格式示例:

代码块

```
1  {
2    "header": {
3      "seq": 1,
4      "stamp": {
5        "secs": 1622548800,
6        "nsecs": 0
7      },
8      "frame_id": ""
9    },
10   "velocity": {
11     "linear": {
12       "x": 1.0,
13       "y": 0.0,
14       "z": 0.0
15     },
16     "angular": {
17       "x": 0.0,
18       "y": 0.0,
19       "z": 0.0
20     }
21   },
22   "walk_mode": 1,
23   "error_code": 0
24 }
```

5.19.5 机身高度和姿态控制接口

- 接口名称: /hric/robot/float_base_rpyz
- 类型: topic
- 描述: 站立状态下发布腰部rpyz指令, 控制机器人姿态。
- 订阅模块: 机器人管理
- 消息类型: hric_msgs/FloatBaseRPYZ.msg
- 消息频率: 400hz
- 控制范围:

代码块

```
1 height[0.6,0.89]m 默认0.89m
```

```
2 yaw[-1.0,1.0]rad
3 pitch[-0.3,0.4]rad
4 roll[-0.2,0.2]rad
```

- **发布示例：**

```
1 cd ros2ws/
2 source install/setup.bash
3 ros2 topic pub /hric/robot/float_base_rpyz_cmd hric_msgs/msg/FloatBaseRPYZ "{
4   roll: 0.0, pitch: 0.05, yaw: 0, z: 0.88
5 }"
```

5.19.6 机身高度和姿态状态反馈接口

- **接口名称：** /hric/robot/float_base_rpyz_status
- **类型：** topic
- **描述：** 发布站立状态下机器人浮动基座的实际rpyz
- **发布模块：** 运动控制
- **发布频率：** 400hz
- **订阅模块：** 机器人管理、操作
- **消息类型：** hric_msgs/FloatBaseRPYZ.msg
- **消息格式示例：**

代码块

```
1 ---
2 roll: 4.081035528590296e-05
3 pitch: 0.02553808313186995
4 yaw: -0.0002636909484863281
5 z: 0.8828393155853616
```

5.19.7 动作状态反馈接口

- **接口名称：** /hric/robot/action_status
- **类型：** topic
- **描述：** 获取机器人当前动作序号以及当前是否有动作在执行。
- **发布模块：** 运动控制

- **发布频率：** 400hz
- **订阅模块：** 机器人管理、操作
- **消息类型：** hric_msgs/ActionStatus.msg
- **消息格式示例：**

代码块

```
1  ---
2  motion_number: 1
3  is_motion: false
```

5.19.8 x方向速度限制状态反馈接口

- **接口名称：** /hric/robot/get_vel_limit
- **类型：** topic
- **描述：** 获取机器人当前设置的x方向速度限制状态。
- **发布模块：** 运动控制
- **发布频率：** 400hz
- **订阅模块：** 机器人管理、操作
- **消息类型：** hric_msgs/GetVelLimit.msg
- **消息格式示例：**

代码块

```
1  ---
2  x_vel_limit_walk: 1.0
3  x_vel_limit_run: 2.0
```

5.19.9 x方向速度限制设置接口

- **接口名称：** /hric/robot/set_vel_limit
- **类型：** topic
- **描述：** 设置机器人的x方向速度限制，单位：m/s，仅在机器在“ZERO”或“STOP”状态响应，重启失效。
- **发布模块：** 运动控制
- **发布频率：** 400hz
- **订阅模块：** 机器人管理、操作

- 消息类型: hric_msgs/SetVelLimit.msg

- 消息格式示例:

代码块

```
1 cd ros2ws/  
2 source install/setup.bash  
3 ros2 topic pub /hric/robot/set_vel_limit hric_msgs/msg/SetVelLimit "{  
4   x_vel_limit_walk: 0.5, x_vel_limit_run: 1.5  
5   }"
```